

MILLION: MasterIng Long-Context LLM Inference Via Outlier-Immunized KV Product QuaNtization

Zongwu Wang^{1,2,†}, Peng Xu^{1,2,†}, Fangxin Liu^{1,2,*}, Yiwei Hu¹, Qingxiao Sun⁴,
Gezi Li³, Cheng Li³, Xuan Wang³, Li Jiang^{1,2,*}, and Haibing Guan¹

1. Shanghai Jiao Tong University, 2. Shanghai Qi Zhi Institute,

3. Huawei Technologies Co., Ltd, 4. China University of Petroleum-Beijing

*Corresponding Author {wangzongwu, liufangxin, ljiang_cs}@sjtu.edu.cn

Abstract—Large language models (LLMs) are increasingly utilized for complex tasks requiring longer context lengths, with some models supporting up to 128K or 1M tokens. This trend, however, presents significant challenges in inference speed and memory management. The primary bottleneck in long-context LLM inference is the quadratic computational complexity of attention mechanisms, causing substantial slowdowns as sequence length increases. KV cache mechanism alleviates this issue by storing pre-computed data, but introduces memory requirements that scale linearly with context length, hindering efficient LLM deployment. Quantization emerges as a promising approach to address the widening gap between LLM size and memory capacity. However, traditional quantization schemes often yield suboptimal compression results for KV caches due to two key factors: i) On-the-fly quantization and de-quantization, causing significant performance overhead; ii) Prevalence of outliers in KV values, challenging low-bitwidth uniform quantization. To this end, we propose MILLION, a novel quantization framework achieving low-bitwidth KV cache through product quantization. First, we conduct a thorough analysis of KV cache distribution, revealing the limitations of existing quantization schemes. Second, we introduce a non-uniform quantization algorithm based on product quantization, which efficiently compresses data while preserving accuracy. Third, we develop a high-performance GPU inference framework with efficient attention kernel and pipeline design for MILLION that leverages sparse computation and asynchronous quantization, significantly enhancing inference speed. Comprehensive evaluation results demonstrate that MILLION can achieve 4 bits quantization with trivial perplexity and accuracy loss, and achieve 2.09x end-to-end performance gains at 32K context length. Code is released at <https://github.com/ZongwuWang/MILLION>.

I. INTRODUCTION

Large language models (LLMs) have demonstrated remarkable capabilities across various tasks, with recent efforts focusing on expanding their context lengths to further enhance performance. Longer context lengths enable new applications such as long document summarization, retrieval-based question answering for extensive documents, extended multi-turn interactions, and code analysis [1]. This demand has driven significant advancements in long-context models in both industry [2], [3] and academia [1]. While increased contextual capability enhances model intelligence, it also introduces substantial computational demands, scaling quadratically ($O(n^2)$)

with context length. To address this challenge, modern LLM frameworks have adopted the key-value (KV) cache, which reduces the computational complexity to $O(n)$ at the cost of increased memory usage. However, the practice of batch processing requests, while improving tensor processing efficiency, further exacerbates storage overhead as KV caches cannot be shared between multiple requests [4], [5].

KV cache storage scales linearly with context length, often surpassing device memory limits. For example, in the 540B PaLM model, a batch size of 512 and context length of 2048 can result in a 3TB KV cache—three times the model’s parameter size. Additionally, each token generation in the decoding phase requires full KV cache access, creating a major memory bottleneck in long-context scenarios. Recent research has focused on addressing these memory-related challenges in LLMs through three primary approaches: **1) Optimized attention mechanisms:** Techniques such as MQA [6], GQA [7], MLA [8], YOCO [9], and CHAI [10] have been proposed to reduce computational and storage overheads by sharing KVs among attention heads. However, these optimizations often come at the cost of reduced accuracy, with only GQA currently seeing widespread adoption in state-of-the-art LLMs. **2) Sparse attention mechanisms:** These approaches aim to reduce the complexity of attention computations. For example, given the importance of local context, sliding window attention maintains a fixed-size sliding window of recent token states to limit the number of tokens involved in computations [11]. Streaming-LLM further improves on this by introducing the “attention sink” tokens [12], demonstrating that retaining initial token KVs can recover much of the lost performance. More advanced KV sparsity techniques [13]–[18] attempt to identify and remove less important token KVs based on historical attention patterns. However, recent studies [19], [20] suggest that these approaches may suffer from accuracy degradation, as past attention distributions may not reliably predict future attention needs in long-context generation. **3) Key-value quantization:** Quantization, successful in model weight compression [21]–[24], has emerged as a promising technique for KV cache optimization. However, applying weight quantization methods directly to KV caches presents unique challenges: First, the presence of outliers in the KV states of LLMs expands the quantization range, making low-bit quantization

† These authors contributed equally. This work was supported by the National Key Research and Development Program of China (2024YFE0204300), National Natural Science Foundation of China (Grant No.62402311), and Natural Science Foundation of Shanghai (Grant No.24ZR1433700).

difficult. Second, since KVs are dynamically generated during inference, quantization or de-quantization delays can negatively impact real-time performance. According to [25], 4 bit integer dequantization contributes 20% to 90% of the inference latency. Additionally, linear attention mechanisms like Manba [26], RWKV [27] and InfiniTransformer [28] have been developed to further reduce the complexity of attention computations. However, these approaches also tend to trade off some degree of model accuracy for efficiency gains.

Recent works have attempted to address these challenges: QServer [25] proposed SmoothAttention, which implements 4-bit KV cache quantization with the SmoothAttention scale Λ into previous layers' weights to reduce de-quantization overhead, however the accuracy is limited by the hard constraint introduced by the rotary positional embedding. KIVI employs group-wise quantization, adapting to outlier distribution through channel-wise and token-wise quantization of keys and values. KVQuant further refines this approach with outlier isolation and non-uniform quantization, improving low-bitwidth accuracy at the cost of inference performance. Unfortunately, these techniques often degrade inference speed due to the overhead introduced by outlier isolation and non-uniform quantization. The ideal solution would balance these factors without significantly compromising any single aspect.

This work introduces MILLION, a novel quantization scheme for efficient dynamic KV compression in LLMs. MILLION utilizes non-uniform quantization and eliminates de-quantization overhead, integrating seamlessly with a high-performance system optimized for GPU-based inference. First, we thoroughly analyze the distribution patterns of KVs in LLMs, uncovering the limitations of existing quantization methods. Based on these distribution characteristics, we propose a non-uniform quantization scheme using Product Quantization (PQ). By clustering vectors, PQ distributes quantization power unevenly across channels, which helps handle outliers in the KV states. PQ also naturally supports mixed-precision quantization, offering flexibility in balancing performance and accuracy. Finally, we design an efficient GPU inference framework for MILLION, which accelerates inference by leveraging sparse computation and asynchronous quantization, boosting processing speed while reducing memory overhead.

In summary, we make the following contributions:

- **KV distribution analysis:** We perform an in-depth analysis of the distribution characteristics of key-value pairs in LLMs, identifying the limitations of existing quantization methods and highlighting the unique challenges posed by outliers and dynamic KV generation.
- **PQ-based non-uniform quantization:** Building on the insights from the KV distribution analysis, we propose a novel non-uniform quantization scheme based on PQ. This approach leverages clustering to distribute quantization power unevenly across channels, enabling efficient compression while maintaining high accuracy.
- **Efficient GPU inference system:** We design and implement a high-performance GPU-based inference framework optimized for MILLION. This system accelerates

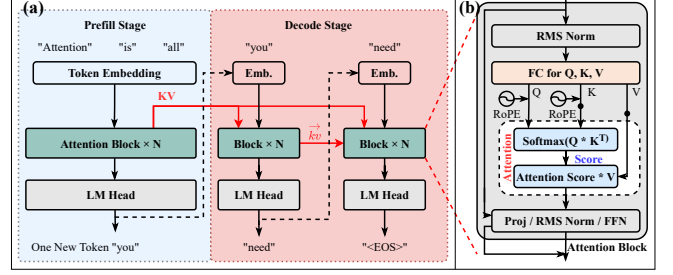


Fig. 1. Overview of transformer model structure. (a) Transformer model consists of two stage, the prefill stage processes the prompt prefilling of all tokens in batches, and the decode stage generates tokens one by one; (2) Details of the attention block.

inference through sparse computation and asynchronous quantization, effectively eliminating de-quantization overhead while ensuring fast, scalable, and memory-efficient processing of long-context inputs.

II. PRELIMINARY

A. Attention-based LLM model.

The Transformer model architecture consists of two main stages: the Prefill stage and the Decode stage, as depicted in Fig. 1(a). In the Prefill stage, the model processes the entire input sequence in batches, and generate the key and value vectors for each prompt tokens simultaneously. The Decode stage operates auto-regressively, generating the output sequence token by token. The auto-regressively interaction is handled by the attention blocks, which is the most important modules in the transformer models. The detailed algorithm of the attention block is depicted in Fig. 1(b). The unique attention mechanism, highlighted with a white background, enables token interactions within the context. This process is formalized as follows:

$$A_n = \text{Softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V \quad (1)$$

Where $Q \in \mathbb{R}^{n_q \times d_k}$, K and $V \in \mathbb{R}^{n_k \times d_k}$ represent the query, key, and value vectors corresponding to each token, d_k denotes the head dimension, n_q and n_k denote the token number of query and key/value for the attention computation, respectively. Due to the causality of the language model, the query vector of each token needs to be feature fusion with all previous key/value vectors. Then the attention scores are utilized to calculate the weighted sum of the value vectors for each token, resulting in the current context state vector. This is followed by feature extraction using additional FFN layers.

B. Integer Quantization

Existing integer quantization maps high-precision numbers to discrete levels, the process can be formulated as:

$$\begin{cases} Q_X = \lceil \frac{X}{s} + z \rceil \\ s = \frac{X_{max} - X_{min}}{q_{max} - q_{min}} \\ z = \lceil q_{min} - \frac{X_{min}}{s} \rceil \end{cases} \quad (2)$$

where X is the floating point tensor, Q_X is its n-bit quantized counterpart, s is the scaling factor and z is the zero point. Thus, the dequantized tensor can be represented as,

$$\hat{X} = Q(X) = (Q_X - z) \cdot s \quad (3)$$

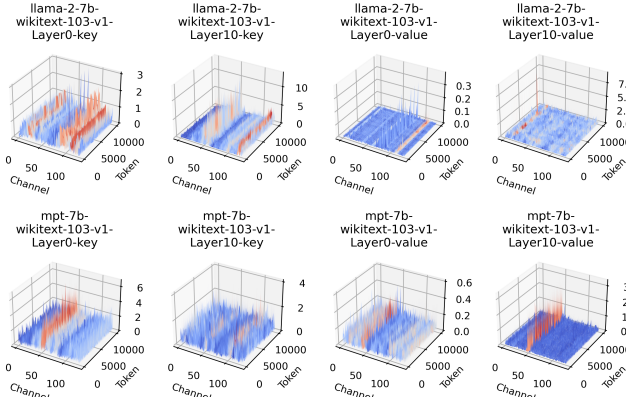


Fig. 2. Magnitude distribution of key and value cache for Llama-2-13B and Falcon-7B.

This is known as asymmetric quantization, where $\mathbf{X}^{max} = \max(\mathbf{X})$, $\mathbf{X}^{min} = \min(\mathbf{X})$, and $q_{max} - q_{min} = 2^n - 1$ for n -bit integer quantization. Fig. 2 can be further simplified to symmetric quantization, where the zero point z is set to 0, and the range is centered around zero with $\mathbf{X}^{max} = -\mathbf{X}^{min} = \max|\mathbf{X}|$. In this case, $q_{max} - q_{min} = 2^n - 2$, as one bit is reserved for the sign.

The de-quantization process of integer quantization needs to be implemented using general-purpose CUDA cores, which are less efficient for this type of operation. This implementation introduces a significant performance bottleneck, resulting in 20% to 90% overhead [25].

C. Product Quantization

Product Quantization (PQ) is an innovative algorithm that revolutionizes high-dimensional vector retrieval. It offers an efficient and scalable solution for large-scale approximate nearest neighbor searches (ANNS) in high-dimensional spaces [29]. The key concept of PQ is the partitioning of high-dimensional vectors into several lower-dimensional subspaces, each independently quantized. This approach significantly reduces the complexity and storage cost of high-dimensional vector similarity operations. According to [30], PQ can achieve 90% memory compression without significant accuracy loss.

The PQ process begins with a training phase, where a codebook is created for each subspace using k-means clustering. Each vector is partitioned and assigned to the nearest centroid in its subspace, with the centroid index stored as a compact code, effectively compressing the data and reducing storage. During the retrieval phase, query vectors are similarly partitioned and quantized. The algorithm approximates the distance by summing precomputed distances between the query subvectors and the centroids of each corresponding subspace in the database. This asymmetric distance computation enables fast ANNS without decoding, achieving both storage efficiency and quick retrieval.

D. Motivation

In this section, we analyze the distribution of KV cache to reveal the limitations of existing low-precision quantization methods. This insight informs our proposed product quantization-based scheme for efficient KV cache compression.

1) *Outliers hinder low-precision quantization.* To explore the root cause of poor performance in existing KV quantization

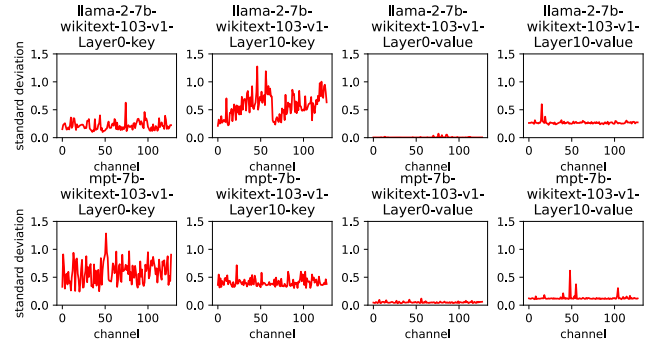


Fig. 3. channel-wise standard deviation distribution of key and value cache for Llama-2-13B and Falcon-7B.

algorithms, we analyzed the KV distributions across different models, as shown in Fig. 2. Our findings reveal that in various models, outliers in the key cache are concentrated in several channels, while the value cache vectors also contain outlier points but without obvious anisotropy in their distribution. As shown in Fig. 2, quantization accuracy depends on the distribution range of the tensor for a given bit width. Concentrated tensor distributions yield higher quantization accuracy. However, outliers significantly expand the numerical distribution range, complicating low-precision quantization. While group-wise and mixed-precision quantization can mitigate outliers, they introduce additional overheads for quantization/de-quantization.

2) *Exploring Opportunities for Non-uniform Quantization.* We further explore the standard deviation distribution of KV across different channels in Fig. 3. Key standard deviations are notably prominent in certain channels, which we term as standard deviation outliers. In contrast, value standard deviations remain relatively small across channels. The channel-wise standard deviation indicates the range of numerical distribution across channels, corresponding to $\mathbf{X}^{max} - \mathbf{X}^{min}$ in Fig. 2. This suggests that integer quantization of keys is more challenging than values, necessitating distinct quantization precisions for different channels.

Given the presence of amplitude and standard deviation outliers in KV cache, we propose using PQ for KV cache compression. PQ splits vectors into multiple subspaces and clusters multiple channels within each subspace for quantization. Channels in the same subspace are uniformly quantized to 2^{nbits} states, where $nbits$ denotes the quantization bits of each subspace vector. Following PQ centroid training principles, channels with higher quantization difficulty will occupy more states in the PQ centroid, inherently supporting mixed precision quantization between channels.

III. EFFICIENT KV CACHE COMPRESSION VIA PQ

A. MILLION Framework Overview

This section illustrates the algorithm framework of the proposed MILLION, which enhances dynamic KV compression using the PQ algorithm. As shown in Figure 4, MILLION consists of three main phases: (1) offline PQ codebook training, (2) online prefill with KV cache quantization, and (3) online decode with KV cache quantization.

Offline PQ Codebook Training. The process begins with offline PQ codebook training of the KV cache, similar to obtaining activation distribution in activation quantization scenarios. During baseline inference, KV cache is sampled, and then

sampled keys and values are used for PQ training to determine the codebooks. Figure 4 a) illustrates this process: sampled key/value vectors are first partitioned into multiple sub-vectors ①, then the k-means algorithm is applied to find centroids for each sub-space ②. After that, the trained centroids are stored in GPU memory as codebooks for future quantization.

Online Prefill with KV Cache Quantization. During the prefill phase, the generated full-precision keys and values are used for attention computation ③, and quantized with the trained codebooks ④. The compressed KV (centroid indices) are then saved into the KV cache. The quantization and de-quantization process can be expressed as Eq. (4) and Eq. (5), respectively.

$$\begin{cases} \Delta_K = \left\| \argmin_{n=1}^M (\|K^i \times C_K^i\|) \right\| \\ \Delta_V = \left\| \argmin_{n=1}^M (\|V^i \times C_V^i\|) \right\| \end{cases} \quad (4)$$

$$\begin{cases} \hat{K} = \left\| \argmin_{n=1}^M \Delta_K^i \otimes C_K^i \right\| \\ \hat{V} = \left\| \argmin_{n=1}^M \Delta_V^i \otimes C_V^i \right\| \end{cases} \quad (5)$$

where M indicates the subspace number of PQ, $\|$ denotes concatenation of M sub-vectors, C_K^i and $C_V^i \in \mathbb{R}^{2^{nbits} \times d_k}$ are the i -th PQ codebooks of K and V . Δ_K^i and $\Delta_V^i \in \mathbb{R}^{n_k}$ are the quantized K and V of the i -th subspace, and $nbits$ denotes the quantization bits for each subspace of PQ, so the 2^{nbits} is the codebook size of each subspace, and the symbol $\otimes$ means column-wise sampling C^i according to the indices of Δ_i . $\hat{K}$ and $\hat{V}$ represent the de-quantized K and V .

Online Decode with KV Cache Quantization. In the decode phase, the current token generates pairwise query, key, and value vectors. Based on the auto-regressive mechanism, the current token needs to compute attention not only with itself but also with all historical tokens ⑥. This phase requires traversing all codes in the KV cache and restoring them to float vectors by looking up the codebooks. The attention computation can be represented as:

$$A_n = \text{Softmax}\left(\frac{q_n \times [\hat{K}_{n-1}^T, k_n^T]}{\sqrt{d_k}}\right) \times [\hat{V}_{n-1}, v_n] \quad (6)$$

Where A_n denotes the attention output of the n -th token. We obtain $\hat{K}$ and $\hat{V}$ by Eq. (5). Lastly, the current key and value are quantized using codebooks and appended to the KV cache ⑦.

B. Efficient Quantization Transformation

While PQ compression reduces KV cache memory demands, quantization and de-quantization overheads (as shown in Eqs. (4) to (6)) still persist during the decoding phase. This section introduces an efficient design to address these inefficiencies.

Attention Computation Transformation. We begin by transforming the attention computation as follows:

$$\begin{aligned} A_n = & \text{OlSoftmax}\left(\frac{q_n \times \left\| \argmin_{i=1}^M C_K^i \otimes \Delta_K^i \right\|^T}{\sqrt{d_k}}\right) \times \left\| \argmin_{i=1}^M \Delta_V^i \otimes C_V^i \right\| \\ & + \text{OlSoftmax}\left(\frac{q_n \times k_n}{\sqrt{d_k}}\right) \times v_n \end{aligned} \quad (7)$$

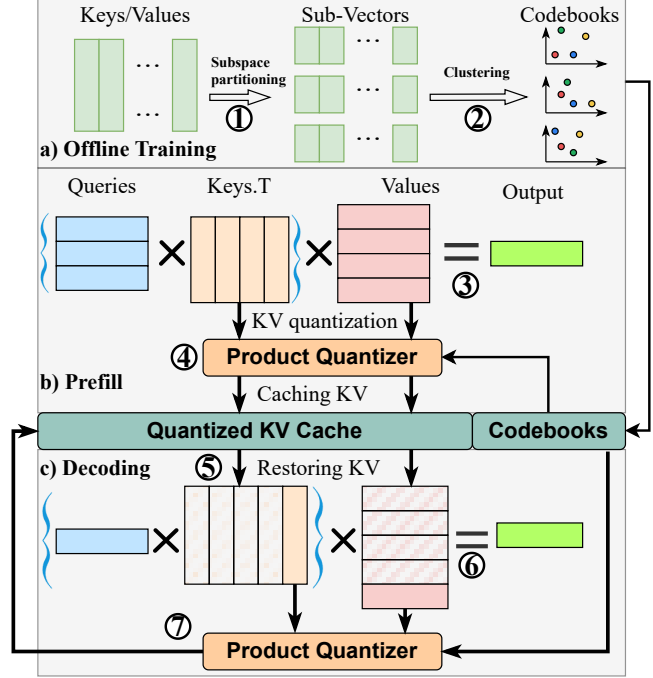


Fig. 4. An overview of MILLION algorithm framework. MILLION's algorithm framework consists of three parts: 1) offline PQ centroid training; 2) Prefill stage with KV quantization; 3) Decode stage with KV quantization.

Here, $\text{OlSoftmax}(\cdot)$ denotes an online softmax operation that splits the attention computation into two parts:

- 1) The current token's attention output to past tokens (first term $\text{OlSoftmax}(\frac{q_n \times \left\| \argmin_{i=1}^M C_K^i \otimes \Delta_K^i \right\|^T}{\sqrt{d_k}}) \times \left\| \argmin_{i=1}^M \Delta_V^i \otimes C_V^i \right\|$)
- 2) The current token's self-attention output (second term $\text{OlSoftmax}(\frac{q_n \times k_n}{\sqrt{d_k}}) \times v_n$)

The second part of the Eq. (7), dealing with the current token's self-attention, thus uses the full-precision key and value. It has $\mathcal{O}(1)$ computational complexity and requires no quantization or de-quantization for subsequent token generation. The first part, on the other hand, addressing attention to past tokens, is computed through two steps: 1) Calculate $q_n \times \left\| \argmin_{i=1}^M C_K^i \otimes \Delta_K^i \right\|^T$ with $\mathcal{O}(1)$ complexity; 2) Sampling the resulting vector with Δ_K , having $\mathcal{O}(n)$ complexity.

Efficient Attention Implementation. Given that Δ_K matrix columns are index vectors, we developed an efficient CUDA kernel. This kernel loads $q_n \times \left\| \argmin_{i=1}^M C_K^i \otimes \Delta_K^i \right\|^T$ as a look-up table into GPU L1 cache for fast random accessing during sampling and obtains the final result through sampling, thus the quantized Δ_K can be computed directly without de-quantization. Besides, the Δ_K matrix is prefetched with float4 data format, which further enhances the efficiency of continuous memory access. For instance, with 8-bit subspace product quantization ($nbits = 8$), each thread in the CUDA kernel can simultaneously load 16 indexes into registers in float4 format, and then use these to perform 16 shared memory address lookups in int8 format. Furthermore, the kernel effectively paralleliz table lookup of Δ_K and de-quantization of Δ_v over CUDA thread blocks inspired by flash decoding.

Asynchronous Quantization Strategy. As of this point

TABLE I
DETAIL MODEL INFORMATION FOR EVALUATION.

	Parameters	positional embedding	sequence length
GPT2-xl	1.5 B	Absolute	1024
LLaMA-2-7B	7B	RoPE	4096
MPT-7B	7B	ALiBi	2048
Longchat-7B	7B	RoPE	32K
Yarn-LLaMA-2-7B	7B	RoPE	128K

TABLE II
PERPLEXITY EVALUATION RESULTS ON VARIOUS MODELS ACROSS DIFFERENT DATASETS.

Method	GPT2-xl		LLaMA-2-7B		MPT-7B	
	Wikitext-2	PTB	Wikitext-2	PTB	Wikitext-2	PTB
baseline	17.41	21	5.12	28.31	7.68	9.98
KVQuant-3b	18.59	23	11.21	12323.75	27681432	3E+07
KVQuant-3b-1%	18.24	22.35	5.22	24.34	7.826	10.14
MILLION-3b	17.6	21.14	5.2	29.55	7.79	10.08
KVQuant-4b	18.156	22.32	6.99	102.21	24043348	3E+07
KVQuant-4b-1%	18.108	22.15	5.14	25.86	7.71	10.02
MILLION-4b	17.57	21.12	5.21	29.56	7.75	10.03

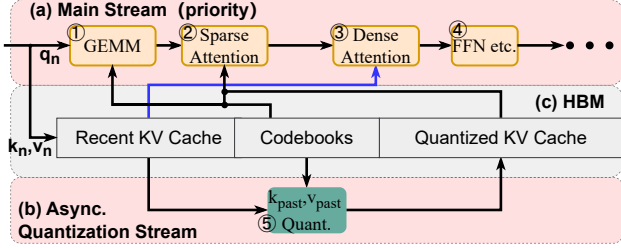


Fig. 5. Efficient Attention Implementation of MILLION with Asynchronous Quantization.

of introducing, quantization of k_n and v_n is still performed sequentially before the attention mechanism. However, cached codes are not needed until next token generation, indicating the computational independence of these two steps. Given the fact that decoding phase is memory-bounded, we assign quantization to a lower-priority CUDA stream to enable asynchronous execution of quantization and the subsequent processing, aiming at maximizing GPU utilization.

C. MILLION Dataflow

The dataflow of MILLION for LLM decoding is illustrated in Fig. 5. To minimize the impact of (de-)quantization on inference performance, MILLION employs two CUDA streams: 1) A high-priority main (CUDA) stream for attention computation; 2) A low-priority auxiliary (CUDA) stream is responsible for key and value quantization.

The process begins with the hidden state undergoing linear transformation to generate the query, key, and value for the current token, which serve as inputs to the attention layer. As analyzed in the previous section, the query performs a GEMM operation with the codebooks to calculate distances from each query to all centroids in the codebooks ①. The resulting distance vector is then used for sparse attention calculation with the quantized keys and values ②. Concurrently, the query undergoes dense attention computation with the recent KV cache (full precision) ③. Sparse and dense attention outputs are combined through online softmax and fed into subsequent operators for further inference ④. To avoid blocking the the main stream, keys, and values generated during decoding are temporarily stored in the recent KV cache. When GPU utilization of the main stream’s CUDA kernel is low, the asynchronous quantization stream initiates the quantization kernel. It retrieves full-precision KV from the static recent KV cache for batch quantization ⑤. This design efficiently balances the computational demands of attention processing and quantization, optimizing overall inference performance.

IV. EXPERIMENTS

A. Experimental Setup

Models. Given the critical role of positional embeddings in the query and key components of LLMs, we conduct extensive experiments across various models employing different positional encoding techniques. Detailed specifications of these models are presented in Table I.

Tasks. We evaluate the quantization performance of MILLION across two distinct task categories: First, to assess model accuracy, we employ Perplexity (PPL), widely regarded as the most stringent metric, to compare the accuracy of various quantization schemes during the prefill stage. We conduct the PPL experiments on both Wikitext-2 and PTB datasets. Second, we conduct generation tasks using LongBench [31] for long context scenarios. This comprehensive assessment covered a range of linguistic challenges across various context lengths.

Baselines. We use the LLM model in float16 format as the baseline. We also compare our method with two state-of-the-art KV cache quantization works: KIVI [32] and KVQuant [33]. For these comparisons, we use the quantization bit widths recommended in their original papers. KIVI employs 2-bit and 4-bit quantization, while KVQuant adopts 3-bit and 4-bit quantization.¹

B. Accuracy Results

PPL Evaluation. Table II compares the perplexity between KVQuant and MILLION. We reproduce KVQuant’s evaluation on the Llama-2 model and extended the comparison to various models and datasets. KVQuant uses non-uniform quantization for all configurations in Table II. The results show that KVQuant, despite combining key/value quantization in different dimensions with non-uniform quantization, still incurs significant PPL loss. In contrast, full-precision sparse storage of 1% outliers achieves near-lossless performance at 3 bits. This demonstrates the impact of outliers on low-bit-width quantization. While isolating outliers for mixed-precision quantization effectively reduces KV cache quantization bit-width, it has drawbacks. This approach leads to unaligned memory access and introduces performance bottlenecks due to online sparse encoding, decoding, and computation. MILLION, employing uniform quantization, outperforms KVQuant in both 4-bit and 3-bit quantization². Its accuracy is slightly lower than KVQuant’s 1% outlier mixed-precision scheme. This indicates

¹We conclude that the PPL of KIVI is huge and therefore omit it from the PPL experiment.

²We scanned different combinations of M and $nbits$ of PQ, and in this paper, the experiments adopted the best accuracy conditions of 3b and 4b, which were (64, 8) and (32, 12), respectively.

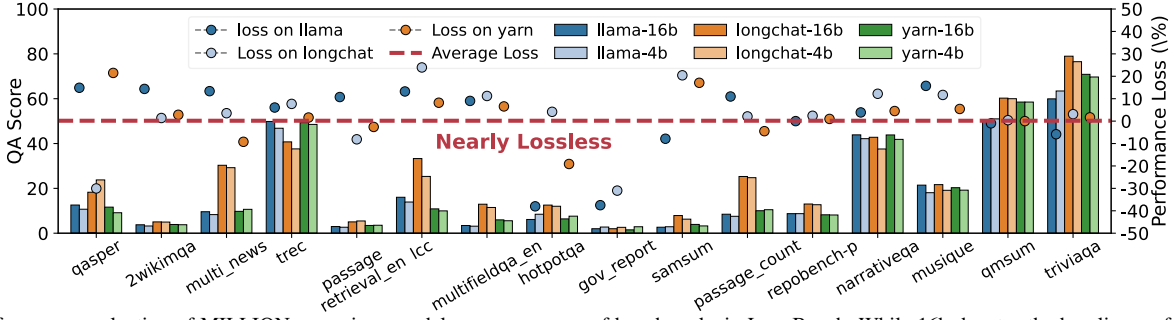


Fig. 6. Performance evaluation of MILLION on various models across a range of benchmarks in LongBench. While 16b denotes the baseline performance with fp16 KV cache, and the 4b depicts the MILLION performance of 4 bits KV quantization.

TABLE III

THE INFLUENCE OF 1% OUTLIERS ON PPL ON KVQUANT AND MILLION

	kv-3b	kv-3b -1%	sensitivity -3b	kv-4b	kv-4b -1%	sensitivity -4b
KVQuant	11.21	5.22	53.4%	6.99	5.14	26.5%
MILLION	5.2	5.22	-0.38%	5.21	5.18	0.58%

TABLE IV

TPOT (MS) COMPARISON OF KV QUANTIZATION METHODS.

Prefill Length	1K	2K	4K	8K	16K	32K
Baseline(fp16)	32.53	35.64	42.04	54.83	80.49	132.97
KIVI(4b)	46.69	46.88	46.92	47.86	OOM	OOM
KVQuant(4b)	75.73	73.92	75.34	74.90	78.17	90.16
MILLION(4b)	30.36	31.57	34.05	38.34	46.53	63.41

that MILLION effectively handles low-bit-width quantization with outliers while maintaining scheme efficiency.

LongBench Evaluation. Figure 6 shows MILLION performance on various models across LongBench benchmarks, the residual block size is set to 0 for stress evaluation. While 4-bit quantization shows some performance degradation compared to full-precision 16-bit models, it maintains sufficient accuracy for many applications. Specifically, MILLION with 4-bit KV cache quantization resulted in a score drop of 0.95 for llama-2-7b and 0.93 for longchat-7b. Notably, MILLION improved the score of yarn-llama-2-7b (with a 128K context) by 0.45, demonstrating its effectiveness for models with the longest contexts.

Outlier-immune Study. To verify MILLION’s sensitivity³ to outliers, we conduct additional experiments, as shown in Table III. We identified the top 1% of outliers in the KV cache based on data distribution and stored them in a sparse full-precision format. After applying MILLION to the filtered KV cache on LLMs, the PPL improved by only -0.38% and 0.58% in the 3-bit and 4-bit quantization scenarios, respectively. This marginal improvement suggests that MILLION is naturally robust to outliers, and incorporating sparse outlier handling offers little benefit, *while significantly degrading end-to-end system performance.*

C. System Performance

Generation Speed: We compare the performance of different KV quantization schemes by running Llama2-7b inference tasks on an A40 GPU. We assessment the average time per output token (TPOT) across different prefill lengths with 100 tokens generation. All schemes use 4-bit KV quantization,

³Sensitivity represents the proportion of PPL reduction caused by considering 1% sparse outliers.

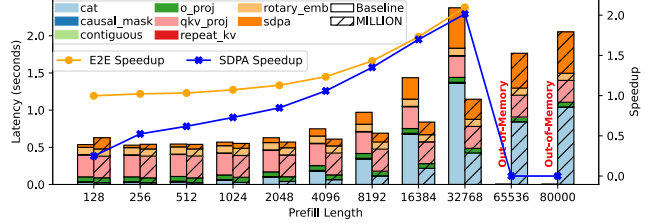


Fig. 7. Latency breakdown analysis. MILLION significantly outperforms the baseline on two KV access operators, the performance margins increasing with the severity of the memory bottleneck.

with results shown in Table IV. KIVI, which uses a simpler quantization scheme, outperforms the baseline at 8K context lengths. In contrast, KVQuant’s non-uniform quantization significantly slows inference, and the outlier sparsity scheme further reduces performance. MILLION reduces computational complexity while maintaining high compression, delivering superior performance even at 1K context length.

Latency Breakdown: Fig. 7 shows the latency breakdown of each operator to highlight the performance gains in the MILLION system. Compared to the baseline, MILLION achieves significant performance improvements in two operators: Scaled Dot Product Attention (SDPA), which performs the attention operation, and Concatenate (cat), which manages the KV cache. This demonstrates that MILLION suppresses the access bottleneck in the generation phase by compressing the KV bitwidth, while reducing the computational complexity of attention further enhances performance. In contrast, other compression schemes incur higher computational overhead due to de-quantization. Besides, Fig. 7 shows the SDPA and end-to-end performance gains at different context lengths. The speedup of MILLION increases with context length, reaching 2.01x and 2.09x in attention and end-to-end acceleration ratios, respectively, at 32K context length.

V. CONCLUSION

We present MILLION, a novel and efficient PQ-based quantization framework for LLM inference. MILLION efficiently compresses data while preserving accuracy. It incorporates asynchronous quantization, enhancing inference speed and reducing additional operations caused by hardware-inefficient quantization/dequantization for KV cache compression. Experiments demonstrate that MILLION achieves better model performance while maintaining low latency.

REFERENCES

- [1] Y. Chen, S. Qian, H. Tang, X. Lai, Z. Liu, S. Han, and J. Jia, "Longlora: Efficient fine-tuning of long-context large language models," *arXiv preprint arXiv:2309.12307*, 2023.
- [2] anthropic, "Introducing claude 3.5 sonnet \ anthropic," <https://www.anthropic.com/news/claude-3-5-sonnet>, (Accessed on 07/09/2024).
- [3] openai, "Models - openai api," <https://platform.openai.com/docs/models>, (Accessed on 07/09/2024).
- [4] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [5] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *arXiv preprint arXiv:2403.02310*, 2024.
- [6] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *arXiv preprint arXiv:1911.02150*, 2019.
- [7] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.
- [8] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo *et al.*, "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.
- [9] Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, Q. Zhang, J. Wang, and F. Wei, "You only cache once: Decoder-decoder architectures for language models," *arXiv preprint arXiv:2405.05254*, 2024.
- [10] S. Agarwal, B. Acun, B. Homer, M. Elhoushi, Y. Lee, S. Venkataraman, D. Papailiopoulos, and C.-J. Wu, "Chai: Clustered head attention for efficient llm inference," *arXiv preprint arXiv:2403.08058*, 2024.
- [11] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.
- [12] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv preprint arXiv:2309.17453*, 2023.
- [13] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [14] D. Yang, X. Han, Y. Gao, Y. Hu, S. Zhang, and H. Zhao, "Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference," *arXiv preprint arXiv:2405.12532*, 2024.
- [15] J. Dai, Z. Huang, H. Jiang, C. Chen, D. Cai, W. Bi, and S. Shi, "Corn: Cache optimization with recent message for large language model inference," 2024. [Online]. Available: <https://arxiv.org/abs/2404.15949>
- [16] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "Snapkv: Llm knows what you are looking for before generation," *arXiv preprint arXiv:2404.14469*, 2024.
- [17] M. Adnan, A. Arunkumar, G. Jain, P. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.
- [18] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 692–705.
- [19] J. Y. Yang, B. Kim, J. Bae, B. Kwon, G. Park, E. Yang, S. J. Kwon, and D. Lee, "No token left behind: Reliable kv cache compression via importance-aware mixed precision quantization," *arXiv preprint arXiv:2402.18096*, 2024.
- [20] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference," *arXiv preprint arXiv:2406.10774*, 2024.
- [21] V. Egiazarian, A. Panferov, D. Kuznedelev, E. Frantar, A. Babenko, and D. Alistarh, "Extreme compression of large language models via additive quantization," *arXiv preprint arXiv:2401.06118*, 2024.
- [22] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *arXiv preprint arXiv:2404.00456*, 2024.
- [23] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [24] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," 2024. [Online]. Available: <https://arxiv.org/abs/2306.00978>
- [25] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.
- [26] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.
- [27] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, S. Biderman, H. Cao, X. Cheng, M. Chung, M. Grella *et al.*, "Rwkv: Reinventing rnns for the transformer era," *arXiv preprint arXiv:2305.13048*, 2023.
- [28] T. Munkhdalai, M. Faruqui, and S. Gopal, "Leave no context behind: Efficient infinite context transformers with infini-attention," *arXiv preprint arXiv:2404.07143*, 2024.
- [29] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE transactions on pattern analysis and machine intelligence*, vol. 33, no. 1, pp. 117–128, 2010.
- [30] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa *et al.*, "Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search," in *NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 2022, pp. 177–189.
- [31] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou *et al.*, "Longbench: A bilingual, multitask benchmark for long context understanding," *arXiv preprint arXiv:2308.14508*, 2023.
- [32] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *arXiv preprint arXiv:2402.02750*, 2024.
- [33] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," 2024. [Online]. Available: <https://arxiv.org/abs/2401.18079>